

Function Call Diagnostic Test

Name (caps) _____ Student ID (9 digits)

--	--	--	--	--	--	--	--

Lab Time _____ Desk Number: Desk- _____

How to work out what a code fragment does: draw a table

The simplest and most reliable way to work out what a code fragment does (and to understand) is to draw a table. This is called tracing or dry running code, Let's work out what this fragment does when a function call `joinStrings ()`; is executed:

```
public static void joinStrings ()
{
    String s = concat("x", "o");           // L1
    System.out.println(s);                 // L2
    return;
}

public static String concat (String s1, String s2) // L3
{
    String r = s1 + s2;                     // L4
    return r;                               // L5
}
```

For each call to a method we create a new table to record the execution of that method. So when we call `joinStrings` we start a new table. We give the details of the call, so what exactly we are dry running in the first row and the headings in the next. The headings include a line number column, a column for each local variable, a column for each method call and a printed column (if it prints anything).

JoinStrings ()			
LINE	s	concat("x", "o") =?	printed

We then execute Line L1 filling in the next row of the table. However line L1 calls a method so we note the precise details of the call here, but then will need to dry run the new call.

JoinStrings ()			
LINE	s	concat("x", "o") =?	printed
L1		concat("x", "o") =?	

So we pause filling out this table and start a new table to work out what the call `concat("x", "o")` does (we will return to this table once that is done. Note that the header line has the exact details of the call including the arguments passed (here "x" and "o"). Again each variable (including the arguments) is allocated a column. As this method returns a result we add a column indicating what it returns.

concat("x", "o")				
LINE	s1	s2	r	return r

Function Call Diagnostic Test

We now execute line L3. It puts the arguments passed into the formal parameter variables.

concat("x", "o")				
LINE	s1	s2	r	return r
L3	"x"	"o"		

Line L4 then concatenates the values stored and puts the result in r.

concat("x", "o")				
LINE	s1	s2	r	return r
L3	"x"	"o"		
L4			"xo"	

Finally executing this method line L5 returns the value in r, so returns the String "xo".

concat("x", "o")				
LINE	s1	s2	r	return r
L3	"x"	"o"		
L4			"xo"	
L5				return "xo"

The execution of this method call is now over. It took as arguments the Strings "x" and "o" and returned the String "xo". We can now use that returned value to complete the original table for joinStrings completing what line L1 does. The call to concat evaluates to "xo" so we fill that in. That result is then stored in s, so we place the value in that column too.

JoinStrings ()			
LINE	s	concat("x", "o") =?	printed
L1	"xo"	concat("x", "o") =? "xo"	

We now move to executing line L2. It just prints the value stored in s (note no quotes are printed).

JoinStrings ()			
LINE	s	concat("x", "o") =?	printed
L1	"xo"	concat("x", "o") =? "xo"	
L2			xo

Overall what is printed is:

xo

If there had been a second call to concat then we would have started a second table to determine what that call does, so with the new arguments.

Function Call Diagnostic Test

The results do not count for anything. This test is to allow the ECS401 team to diagnose if you need help to ensure they do not fall behind in learning to program. Understanding how for loops work is critical so if you have any problems we want to sort them out now. You must show all your working in the following. No correct table (exactly the same)... no marks.

1. What is printed after a method call `squarePrint()`;

```
public static void squarePrint ()
{
    int answer = square(3);           // L1
    System.out.println(answer);       // L2
    answer = square(2);               // L3
    System.out.println(answer);       // L4

    return;

public static int square (int s)      // L5
{
    int res = s * s;                 // L6
    return res;                      // L7
}
```

Complete the dry run tables overleaf (you may not need all rows or all tables):

squarePrint ()				
LINE	answer	square(3) =?	square(2) =?	printed

square (3)			
LINE	s	res	return res

square (2)			
LINE	s	res	return res

Printed

Function Call Diagnostic Test

2. What is printed after a method call `squarePrint2()`;

```
public static int squarePrint2 ()
{
    int x = 2;                // L1
    x = squarePlus1(x);       // L2
    x = squarePlus1(x);       // L3
    System.out.println(x);    // L4

    return x;                 // L5
}

public static int squarePlus1 (int s)    // L6
{
    int sp1 = (s * s) + 1;              // L7
    return sp1;                         // L8
}
```

Complete the dry run tables (you may not need all rows or all tables):

squarePrint2 ()				
LINE	x	squarePlus1(x) =?	return x	printed

squarePlus1 (2)			
LINE	s	sp1	return sp1

squarePlus1 (5)			
LINE	s	sp1	return sp1

Printed

Function Call Diagnostic Test

3. What is printed after a method call `x6()` ;

```
public static void x6 ()
{
    String s = "2";           // L1
    String t = trip(s);       // L2
    String d = doub(t);       // L3
    System.out.println(t + ":" + d); // L4
    return;
}

public static String doub (String d) // L5
{
    String r = (d + d);       // L6
    return r;                 // L7
}

public static String trip (String t) // L8
{
    String r = (t + t + t);   // L9
    return r;                 // L10
}
```

Complete the dry run tables overleaf (you may not need all rows or all tables):

x6 ()						
LINE	s	t	d	trip(s) =?	doub(d)=?	printed

trip ("2")			
LINE	t	r	return r

doub ("222")			
LINE	d	r	return r

Printed

Function Call Diagnostic Test

4. What is printed after a method call `f()` ;

```
public static void f ()
{
    String s1 = "b";           // L1
    String s2 = "a";           // L1
    String s3 = comb(s2, s1);   // L2
    System.out.println(s2 + s1 + "-" + s3); // L4
    return;
}

public static String comb (String s1, String s2) // L5
{
    String r1 = (s1 + s2);      // L6
    String r2 = (s2 + s1);      // L7
    System.out.println(r2);     // L8
    return (r1 + r2);           // L9
}
```

Complete the dry run tables overleaf (you may not need all rows or all tables):

f ()					
LINE	s1	s2	s3	comb(s2,s1) =?	printed

comb ("a","b")						
LINE	s1	s2	r1	r2	return (r1+r2)	printed

Printed